

Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Faculty of Computer Science & Engineering

STREAMIFY

Console-Based Video Streaming Platform

Semester Project Documentation

CS-112 — Object Oriented Programming and Design

GitHub Repo: <https://github.com/shahan-giki/Streamify.git>

Session: Spring 2026
Instructor: Lect. Said Nabi
Department: Software Engineering
Degree: BS Software Engineering

#	Student Name	Reg. Number	Degree
1	M. Mudassar Shehbaz	2024425	BSSWE
2	Shahan Ali	2024572	BSSWE
3	Yahya Khalid	2024664	BSSWE
4	Syed Hamza Samee	2024705	BSSWE

1. Executive Summary

Streamify is a console-based video streaming platform developed as a semester project for CS-112 Object Oriented Programming and Design. The application simulates a real-world video streaming service with two primary user roles: Creators, who upload and manage video content, and Users, who browse, watch, and interact with that content.

The system is implemented in C++ and demonstrates the application of core Object-Oriented Programming principles including Encapsulation, Inheritance, Polymorphism, and Abstraction, alongside a range of Data Structures and Algorithms including Hash Maps, Stacks, Queues, AVL Trees, Linked Lists, and Graphs.

Persistence is achieved through file-based storage, ensuring user credentials and session data survive across application restarts.

2. Project Overview

Attribute	Details
Project Title	Streamify — Console-Based Video Streaming Platform
Course Code	CS-112
Course Title	Object Oriented Programming and Design
Session	Spring 2026
Instructor	Lect. Said Nabi
Language	C++
Architecture	Console / Terminal Application
Persistence	File-based (users.txt)

3. Main Features

3.1 Authentication & User Management

The platform provides a secure, file-backed authentication system supporting both registration (sign-up) and login. A user's role — either Creator or User — is stored persistently and determines the menu and privileges available after login.

3.2 Video Uploading & Library Management

Creators can upload videos from a local folder by selecting files through an interactive directory browser. All uploaded videos are stored in a custom Hash Map-based Video Library that supports $O(1)$ average-case lookup and insertion.

3.3 Video Viewing & Interaction

Registered Users can browse the video library, watch a selected video, leave comments, and like videos. Interaction data (likes and comments) is stored inside each Video object and is visible to Creators when they view their uploaded content.

3.4 Watch History

Each User session maintains a Stack-based Watch History. Every video watched is pushed onto the stack, enabling the user to review previously watched content in reverse chronological order (most recently watched first).

3.5 Playlist Management

Users can build a personal Queue-based Playlist by enqueueing videos. Videos play in FIFO order — the first added is the first to play — mimicking a standard media queue.

3.6 Category Management

Creators can create content categories using an AVL Tree-based Category Manager. Videos (identified by file path) can be assigned to one or more categories. The AVL Tree guarantees $O(\log n)$ insertion and lookup.

3.7 Video Connections / Related Videos

The platform uses a directed Graph to model relationships between videos. After upload, the system builds sequential edges between all video nodes, enabling a Creator to visualise how videos are connected and which videos can be recommended as follow-ups.

4. User Roles & Requirements

4.1 Creator

- Can register and log in to the platform.
- Can upload video files from a local folder to the shared Video Library.
- Can view all videos currently stored in the Video Library along with their like counts and comments.
- Can create new content categories using the Category Management system.
- Can assign uploaded videos to existing categories.
- Can browse videos within a specific category.
- Can view the video connection graph to understand relationships between uploaded content.

4.2 User

- Can register and log in to the platform.
- Can view all videos available in the Video Library.
- Can watch a selected video (opens using the system default media player).
- Can like and comment on a video after watching it.
- Can review their Watch History (Stack-based, most-recent-first).
- Can add videos to a personal Playlist and play them in FIFO order.

5. Object-Oriented Programming Concepts

5.1 Encapsulation

Every class in the system hides its internal data behind private access specifiers and exposes a controlled public interface. Examples include:

- Video: title, filePath, uploader, likes, and comments are private; accessed through getters and setters.
- VideoLibrary: the internal hash table array and hash function are private; the public interface exposes insert(), find(), remove(), and iterate().
- CategoryTree: the AVL node structure and all tree operations are private; the public API exposes addCategory(), addFilePathToCategory(), and displayCategoryVideos().
- AuthSystem: the user map, file path, and current session data are private; authentication is performed solely through interactiveSignUp() and interactiveLogin().

5.2 Abstraction

The Account class is an abstract base class containing a pure virtual function displayMenu(). This enforces the contract that any concrete account type must provide its own menu implementation, while hiding all role-specific logic from the base class.

5.3 Inheritance

Two concrete classes inherit from Account:

- Creator (public Account) — inherits the username/password/role data and adds references to VideoLibrary, CategoryTree, and VideoGraph.
- User (public Account) — inherits base account data and adds a WatchHistory (Stack) and a Playlist (Queue) by composition.

5.4 Polymorphism

Runtime polymorphism is achieved through the virtual displayMenu() mechanism:

- Account declares displayMenu() as a pure virtual function.
- Creator and User override displayMenu() to display role-specific menus.
- In main.cpp, the correct overriding function is invoked directly on the concrete object based on the authenticated role, demonstrating dynamic dispatch.

5.5 Constructors & Destructors

- VideoLibrary defines a constructor that initialises all hash table buckets to nullptr and a destructor that deallocates all HashNode chains, preventing memory leaks.

- CategoryTree defines a constructor initialising the root to nullptr and a destructor that recursively destroys the entire AVL tree.
- Account and its derived classes use parameterised constructors with member initialisation.

5.6 Composition

The User class owns a WatchHistory and a Playlist by value (composition). The lifetime of these objects is tied to the User object — they are created when the user logs in and destroyed on logout.

5.7 File Handling with OOP

The AuthSystem class encapsulates all file I/O. User credentials are persisted to and loaded from users.txt using private helper methods loadFromFile() and saveToFile(), with std::ifstream and std::ofstream respectively.

6. Data Structures & Algorithms

6.1 Hash Map — Video Library (VideoLibrary)

The VideoLibrary implements a custom chaining hash table with a fixed table size of 100 buckets. Each bucket is a singly-linked list of HashNode objects. The hash function converts a string key (video title) into a bucket index, providing $O(1)$ average-case performance for insertion and lookup.

6.2 Hash Map — Authentication System (AuthSystem)

The AuthSystem uses `std::unordered_map<string, string>` to store username-to-password mappings, giving $O(1)$ average lookup during login validation.

6.3 Stack — Watch History (WatchHistory)

The WatchHistory class wraps a `std::vector` used as a LIFO stack. `push()` adds a video title to the top and `display()` iterates in reverse, showing the most recently watched video first. All operations are $O(1)$.

6.4 Queue — Playlist (Playlist)

The Playlist class wraps a `std::vector` used as a FIFO queue. `enqueue()` appends to the back; `dequeue()` removes from the front. This guarantees videos play in the order they were added. All operations are $O(1)$.

6.5 AVL Tree — Category Management (CategoryTree)

The CategoryTree implements a self-balancing AVL tree. Each node stores a category name and a vector of associated file paths. Left and right rotations are applied after each insertion to maintain a height balance factor of at most 1, guaranteeing $O(\log n)$ insertion and lookup.

6.6 Linked List — File Path Storage

Inside each category node of the AVL tree, a `std::vector<string>` acts as a dynamically-growing linked list of file paths. Traversal to list all videos in a category is $O(n)$ where n is the number of paths in that category.

6.7 Graph — Video Connections (VideoGraph)

The VideoGraph maintains an adjacency edge list using two private members: a `vector<string>` of node titles and a `vector<pair<int,int>>` of directed edges. `buildSequentialConnections()` creates edges between consecutive video nodes. `displayGraph()` iterates all edges in $O(E)$ time.

7. System Architecture & Class Descriptions

The following table describes each class in the system, its role, and the OOP/DSA concepts it demonstrates.

Class	Responsibility	OOP Concepts	DSA Used
Account	Abstract base class for all account types	Abstraction, Encapsulation, Polymorphism	N/A
Creator	Manages video upload, categories, and connections	Inheritance, Polymorphism, Composition	HashMap, AVL Tree, Graph
User	Browses videos, manages watch history and playlist	Inheritance, Polymorphism, Composition	Stack, Queue, HashMap
AuthSystem	Handles sign-up, login, logout, and file persistence	Encapsulation, File Handling	Hash Map (unordered_map)
Video	Stores video metadata, likes, and comments	Encapsulation	Vector
VideoLibrary	Stores and retrieves all uploaded videos	Encapsulation	Custom HashMap (chaining)
CategoryTree	Manages categories and their associated file paths	Encapsulation	AVL Tree
VideoGraph	Models and displays connections between videos	Encapsulation	Graph (adjacency edge list)
Playlist	Manages user's ordered video queue	Encapsulation	Queue (vector-backed)
WatchHistory	Tracks videos watched in current session	Encapsulation	Stack (vector-backed)

8. Features to Coding Matrix

#	Feature Name	DSA Concept	Operation	Complexity	Variables / Objects	Functions
1	User Management	Hash Table (unordered_map)	Lookup, Insert	$O(1)$	3	2
2	Watch History	Stack	Push, Display	$O(1)$	1	3
3	Playlist Management	Queue	Enqueue, Dequeue	$O(1)$	1	2
4	Video Library	Custom HashMap (chaining)	Insert, Find, Iterate	$O(1)$ avg	1	3
5	Category Management	AVL Tree	Insert, Traversal	$O(\log n)$	2	3
6	File Path Management	Linked List (vector)	Traversal, Search	$O(n)$	2	2
7	Video Viewing	HashMap & Iteration	Search, Playback	$O(n)$	1	2
8	Video Connections	Graph (adjacency edge list)	Link, Display	$O(n)$	2	1
9	Video Liking	None	Increment Counter	$O(1)$	1	1
10	Video Uploading	File System Interaction	File Path Storage	$O(1)$	1	2
11	Authentication System	Hash Table & File I/O	Lookup, Insert	$O(1)$	2	2
12	Program Control	None	Menu Traversal	$O(1)$	1	0
13	Category-Based Upload	AVL Tree	Insert, Path Traversal	$O(\log n)$	1	2
14	Play from Playlist	Queue	Dequeue	$O(1)$	1	1
15	Display Categories	AVL Tree	Traversal	$O(\log n)$	1	1
16	Authentication Menu	None	Display, Input	$O(1)$	2	2
17	Upload Video from Folder	File System (directory scan)	Directory Traversal	$O(n)$	1	1

9. Detailed Requirements Breakdown

9.1 Authentication & User Management

1. Display a front screen prompting the user to Sign Up, Log In, or Exit.
2. Accept a username, password, and role (creator / user) during sign-up.
3. Validate that the username is not already taken before persisting the record.
4. Store new credentials to users.txt using file I/O.
5. On login, read credentials from users.txt and compare against entered values.
6. Set the authenticated session state (username, role) upon successful login.
7. Provide a logout option that resets session state and returns to the auth screen.

9.2 Video Uploading & Library Management

8. Present the Creator with a folder path prompt to discover uploadable files.
9. List all video files found in the folder with sequential numbering.
10. Allow the Creator to select a video by number and insert it into the VideoLibrary.
11. Prevent duplicate entries by checking the library before insertion.
12. Support all standard video formats found on the file system (mp4, avi, etc.).

9.3 Video Viewing & Interaction

13. Display the full video library with title, uploader, and like count.
14. Allow a User to select a video by number and launch it using the system player.
15. After viewing, prompt the User to optionally leave a comment.
16. After viewing, prompt the User to optionally like the video.
17. Add the watched video to the User's Watch History stack.

9.4 Watch History

18. On each watch event, push the video title onto the WatchHistory stack.
19. Display history in reverse chronological order (most recent first).
20. Show a descriptive message if the watch history is empty.

9.5 Playlist Management

21. Allow a User to browse available videos and enqueue a selected title.
22. Display the current playlist contents in FIFO order.
23. On Play from Playlist, dequeue the front title and open it with the system player.
24. Show a descriptive message if the playlist is empty.

9.6 Category Management

25. Allow a Creator to add a new category name to the AVL Tree.
26. Validate that a category does not already exist before insertion.
27. Allow a Creator to associate an uploaded video (by file path) to an existing category.
28. Display all categories currently stored in the AVL Tree (in-order traversal).
29. Allow a Creator to view all videos associated with a specific category.

9.7 Video Connections

30. Each uploaded video is added as a node in the VideoGraph.
31. After a new upload, rebuild sequential connections between all nodes.
32. Allow a Creator to display the full connection graph showing all edges.

10. Source File Structure

File	Description
main.cpp	Application entry point. Initialises all shared objects, handles authentication loop, and dispatches role-specific menus via polymorphism.
Account.h	Abstract base class. Declares pure virtual displayMenu() and stores username, password, and role.
Creator.h	Derived class for Creator role. Holds references to VideoLibrary, CategoryTree, and VideoGraph; implements all creator actions.
User.h	Derived class for User role. Holds a reference to VideoLibrary and owns a WatchHistory and Playlist by composition.
AuthSystem.h	Encapsulates authentication: sign-up, login, logout, and file-based persistence.
Video.h	Value class representing a single video: title, file path, uploader, likes, and comments.
VideoLibrary.h	Custom chaining HashMap. Maps video title (string) to Video object. Supports insert, find, remove, and iteration.
Category.h	AVL Tree-based category manager. Nodes store category name and a list of associated file paths.
VideoGraph.h	Adjacency edge-list graph. Nodes are video titles; edges represent sequential/related-video connections.
Playlist.h	Queue wrapper (FIFO). Stores video titles for ordered playback.

11. Complexity Analysis Summary

Component	Operation	Time Complexity	Space Complexity
VideoLibrary (HashMap)	Insert / Find	$O(1)$ average	$O(n)$
VideoLibrary (HashMap)	Iterate all videos	$O(n)$	$O(1)$
AuthSystem (unordered_map)	Login / Sign-up	$O(1)$ average	$O(n)$
WatchHistory (Stack)	Push / Display top	$O(1)$	$O(n)$
Playlist (Queue)	Enqueue / Dequeue	$O(1)$	$O(n)$
CategoryTree (AVL Tree)	Insert / Search	$O(\log n)$	$O(n)$
CategoryTree (AVL Tree)	In-order traversal	$O(n)$	$O(n)$
VideoGraph (Graph)	Add node / Add edge	$O(1)$	$O(V+E)$
VideoGraph (Graph)	Display all edges	$O(V+E)$	$O(V+E)$

12. Practical Implementation

```

=====

=== User Authentication ===
1. Sign Up
2. Log In
3. Exit
Enter your choice: 1
Enter username: hamza
Enter password: hamza
Select role (1. Creator 2. User): 1
Sign-up successful! You can now log in.

=== User Authentication ===
1. Sign Up
2. Log In
3. Exit
Enter your choice: 2
Enter username: hamza
Enter password: hamza
Login successful! Welcome, hamza!

```

Creating a creator account

```
===== Creator Menu =====
1. Upload Video from Folder
2. View Uploaded Videos
3. Add Category
4. Upload Video to Category
5. View Category Videos
6. View Video Connections
7. Logout
Enter your choice: 1
Enter folder path: C:\Milk\Shows\Forgotten (2017) [KOREAN] [1080p] [BluRay] [5.1] [YTS.MX]\Forgotten (2017) [WEBRip] [1080p] [YTS.AM]
Videos available for upload:
  1. Forgotten.2017.1080p.WEBRip.x264-[YTS.AM].mp4
Enter the number of the video to upload: 1
Video uploaded successfully: Forgotten.2017.1080p.WEBRip.x264-[YTS.AM].mp4

===== Creator Menu =====
1. Upload Video from Folder
2. View Uploaded Videos
3. Add Category
4. Upload Video to Category
5. View Category Videos
6. View Video Connections
7. Logout
Enter your choice: 2

=== Uploaded Videos ===
- Forgotten.2017.1080p.WEBRip.x264-[YTS.AM].mp4 (Likes: 0)
No comments yet.

===== Creator Menu =====
1. Upload Video from Folder
2. View Uploaded Videos
3. Add Category
4. Upload Video to Category
5. View Category Videos
6. View Video Connections
7. Logout
Enter your choice: 3
Enter category name: Films
Category added: Films
```

Uploading a video and adding it to a category.

```

===== Creator Menu =====
1. Upload Video from Folder
2. View Uploaded Videos
3. Add Category
4. Upload Video to Category
5. View Category Videos
6. View Video Connections
7. Logout
Enter your choice: 7
Logged out successfully.

=== User Authentication ===
1. Sign Up
2. Log In
3. Exit
Enter your choice: 1
Enter username: usr
Enter password: usr
Select role (1. Creator 2. User): 2
Sign-up successful! You can now log in.

=== User Authentication ===
1. Sign Up
2. Log In
3. Exit
Enter your choice: 2
Enter username: usr
Enter password: usr
Login successful! Welcome, usr!

===== User Menu =====
1. View & Watch Videos
2. View Watch History
3. Add to Playlist
4. Play from Playlist
5. View Playlist
6. Logout
Enter your choice: 1
  1. Forgotten.2017.1080p.WEBRip.x264-[YTS.AM].mp4 (Likes: 0)
  2. Heretic.2024.1080p.WEBRip.10Bit.DDP5.1.Atmos.x265-Asiimov.mkv (Likes: 0)
Enter the number of the video to watch:

```

Uploading a video directly into category.

```

2. Heretic.2024.1080p.WEBRip.10Bit.DDP5.1.Atmos.x265-Asiimov.mkv
===== User Menu =====
1. View & Watch Videos
2. View Watch History
3. Add to Playlist
4. Play from Playlist
5. View Playlist
6. Logout
Enter your choice: 4
Now playing from playlist: Forgotten.2017.1080p.WEBRip.x264-[YTS.AM].mp4

===== User Menu =====
1. View & Watch Videos
2. View Watch History
3. Add to Playlist
4. Play from Playlist
5. View Playlist
6. Logout
Enter your choice: 5
Playlist (play order):
  1. Heretic.2024.1080p.WEBRip.10Bit.DDP5.1.Atmos.x265-Asiimov.mkv

===== User Menu =====
1. View & Watch Videos
2. View Watch History
3. Add to Playlist
4. Play from Playlist
5. View Playlist
6. Logout
Enter your choice: 4
Now playing from playlist: Heretic.2024.1080p.WEBRip.10Bit.DDP5.1.Atmos.x265-Asiimov.mkv

```



After a video from the playlist finishes, it is removed and the next video plays.

```

===== User Menu =====
1. View & Watch Videos
2. View Watch History
3. Add to Playlist
4. Play from Playlist
5. View Playlist
6. Logout
Enter your choice: 1
  1. Forgotten.2017.1080p.WEBRip.x264-[YTS.AM].mp4 (Likes: 0)
  2. Heretic.2024.1080p.WEBRip.10Bit.DDP5.1.Atmos.x265-Asiimov.mkv (Likes: 0)
Enter the number of the video to watch: 3
Invalid choice.

===== User Menu =====
1. View & Watch Videos
2. View Watch History
3. Add to Playlist
4. Play from Playlist
5. View Playlist
6. Logout
Enter your choice: 3
  1. Forgotten.2017.1080p.WEBRip.x264-[YTS.AM].mp4 (Likes: 0)
  2. Heretic.2024.1080p.WEBRip.10Bit.DDP5.1.Atmos.x265-Asiimov.mkv (Likes: 0)
Enter the number of the video to add to playlist: 1
Added to playlist: Forgotten.2017.1080p.WEBRip.x264-[YTS.AM].mp4

===== User Menu =====
1. View & Watch Videos
2. View Watch History
3. Add to Playlist
4. Play from Playlist
5. View Playlist
6. Logout
Enter your choice: 3
  1. Forgotten.2017.1080p.WEBRip.x264-[YTS.AM].mp4 (Likes: 0)
  2. Heretic.2024.1080p.WEBRip.10Bit.DDP5.1.Atmos.x265-Asiimov.mkv (Likes: 0)
Enter the number of the video to add to playlist: 2
Added to playlist: Heretic.2024.1080p.WEBRip.10Bit.DDP5.1.Atmos.x265-Asiimov.mkv

```

Placing videos in a playlist

```

2. View Watch History
3. Add to Playlist
4. Play from Playlist
5. View Playlist
6. Logout
Enter your choice: 3
  1. Forgotten.2017.1080p.WEBRip.x264-[YTS.AM].mp4
  2. Heretic.2024.1080p.WEBRip.10Bit.DDP5.1.Atmos.
Enter the number of the video to add to playlist:
Added to playlist: Heretic.2024.1080p.WEBRip.10Bit

===== User Menu =====
1. View & Watch Videos
2. View Watch History
3. Add to Playlist
4. Play from Playlist
5. View Playlist
6. Logout
Enter your choice: 5
Playlist (play order):
  1. Forgotten.2017.1080p.WEBRip.x264-[YTS.AM].mp4
  2. Heretic.2024.1080p.WEBRip.10Bit.DDP5.1.Atmos.

===== User Menu =====
1. View & Watch Videos
2. View Watch History
3. Add to Playlist
4. Play from Playlist
5. View Playlist
6. Logout
Enter your choice: 4
Now playing from playlist: Forgotten.2017.1080p.WEBRip.x264-[YTS.AM].mp4

```



Playing video from a playlist


```

===== User Menu =====
1. View & Watch Videos
2. View Watch History
3. Add to Playlist
4. Play from Playlist
5. View Playlist
6. Logout
Enter your choice: 1
  1. Forgotten.2017.1080p.WEBRip.x264-[YTS.AM].mp4 (Likes: 0)
  2. Heretic.2024.1080p.WEBRip.10Bit.DDP5.1.Atmos.x265-Asiimov.mkv (Likes: 1)
Enter the number of the video to watch: 1
Now watching: Forgotten.2017.1080p.WEBRip.x264-[YTS.AM].mp4
Would you like to comment on this video? (yes/no): yes
Enter your comment: incredible film
Comment added.
Do you want to like this video? (yes/no): yes
Video liked!

```

Netflix 오리지널 영화

Playing a video from the system, commenting and liking the video

```

===== User Menu =====
1. View & Watch Videos
2. View Watch History
3. Add to Playlist
4. Play from Playlist
5. View Playlist
6. Logout
Enter your choice: 2
Watch History (most recent first):
  1. Forgotten.2017.1080p.WEBRip.x264-[YTS.AM].mp4
  2. Heretic.2024.1080p.WEBRip.10Bit.DDP5.1.Atmos.x265-Asiimov.mkv

```

Watch history

13. Conclusion

Streamify successfully demonstrates the implementation of core Object-Oriented Programming principles alongside practical Data Structures and Algorithms in a coherent, real-world application. The system cleanly separates concerns: each class has a single, well-defined responsibility, all internal data is properly encapsulated, and role-specific behaviour is achieved through inheritance and polymorphism rather than conditional logic scattered across the codebase.

The integration of multiple data structures — a custom chaining HashMap for $O(1)$ average video lookups, an AVL Tree for $O(\log n)$ category management, a Stack for intuitive watch history, a Queue for ordered playlist playback, and a Graph for video relationship modelling — demonstrates the practical impact that choosing the right data structure has on both correctness and performance.

File-based persistence ensures the system behaves like a real application across sessions, reinforcing concepts of file I/O with OOP. Overall, Streamify serves as a comprehensive illustration of how software engineering principles learned in CS-112 translate into a working, structured, and maintainable software product.